MCP with AI Agents

Welcome and Overview

Day 1 - Session 1

KavinSchool

Prerequisites

Confirm these prerequisites:

- Orchestration Fundamentals for Agentic Development or equivalent experience
- Claude Code or GitHub Copilot in VS Code installed and licensed
- Permission to use Model Context Protocol (MCP) in your environment
- Terminal workflow and application programming interface (API) experience
- Basic TypeScript familiarity
- The course setup and companion repositories available locally

Environment Validation

Before we connect any models, let's ensure our environment is ready.

- **Check Node Version:** `node --version` (needs v18+)
- **Check Python Version:** `python --version` (needs 3.10+)
- **Host Authentication:** Ensure you are logged into Claude Code (`claude login`) or GitHub Copilot in VS Code.
- **Common Issues:** If `mcp` commands fail, check your PATH and restart your terminal or VS Code.

> Validating these now ensures smooth execution during the hands-on lab.

Introduce Yourself

In one chat message, type:

1. Your name
2. Your location or time zone
3. Your role
4. Your total experience in IT and at your current organization

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
>
> *Wait for the countdown: 3... 2... 1... BLAST*

A Little Beyond Work

Choose one prompt. Share only what feels comfortable.

01

Bucket List

Name one to three experiences you hope to have.

02

Recent Trip

Share a place you visited and one detail that stayed with you.

03

Memorable Moment

Share a recent moment that made you proud, grateful, or happy.

60 seconds: Share in chat, speak when invited, keep a private note, or pass.

Your MCP Experience

In one chat message, type:

1. **Your current MCP experience** (new to MCP, configured a server, or built a custom server)
2. **Your main focus** (application development, platform integration, security, or technical leadership)
3. **One MCP connection or security question** you want answered by this course

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
>
> *Wait for the countdown: 3... 2... 1... BLAST*

Your MCP Experience: What We Learned

Your responses help us emphasize:

- **Your starting point:** MCP architecture, pre-built server configuration, or custom server development
- **Your focus area:** Client setup, tool design, security boundaries, or enterprise deployment
- **Your key questions:** Discovery, invocation, validation, allowlists, audit evidence, and gateway controls

> 💬 **Compare your response with these themes and note one topic to revisit.**

What We'll Cover

1. Welcome and Overview of MCP
2. Connecting Models to External Data
3. Live Demo: Playwright MCP
4. Demo: Configuring Pre-built Servers
5. Kahoot Quiz 1

Welcome and Overview of MCP

Connecting AI to external capabilities

KavinSchool

What is MCP?

Model Context Protocol (MCP) provides a standardized way to connect AI models to external data sources and tools.

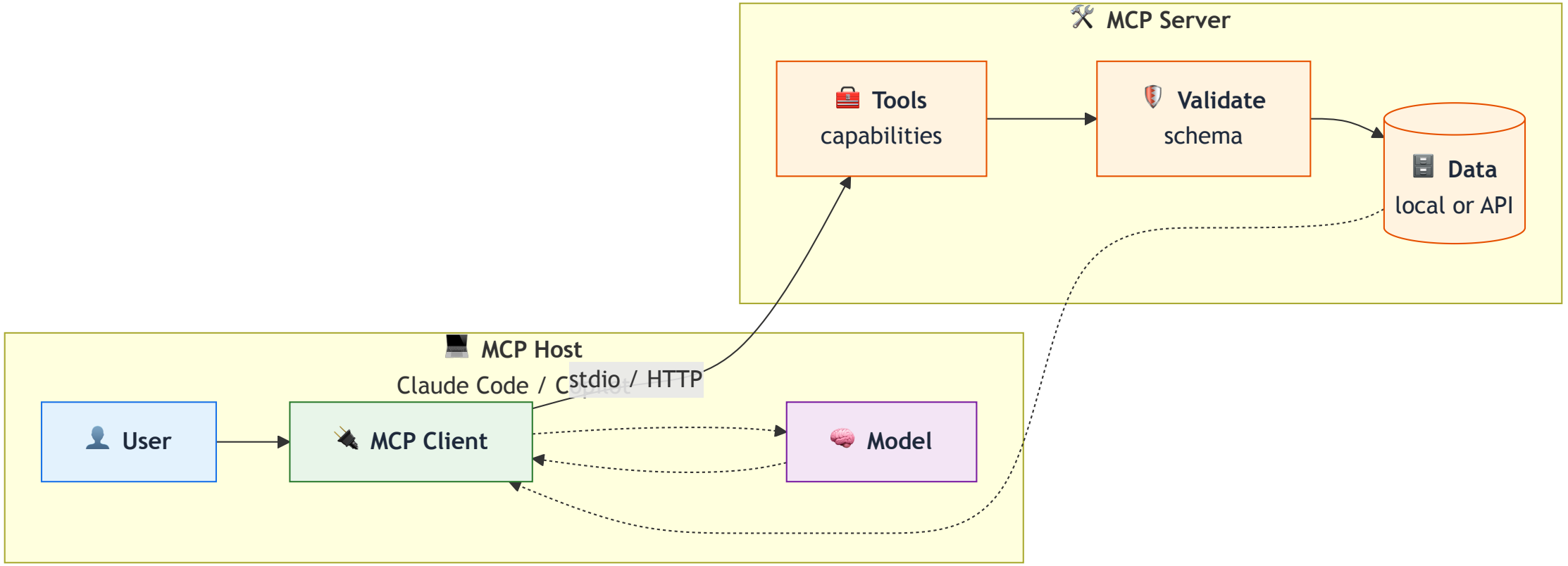
- Replaces bespoke integrations
- Provides a common interface for discovery and invocation
- Allows models to access local files, APIs, and databases securely

Architecture Overview

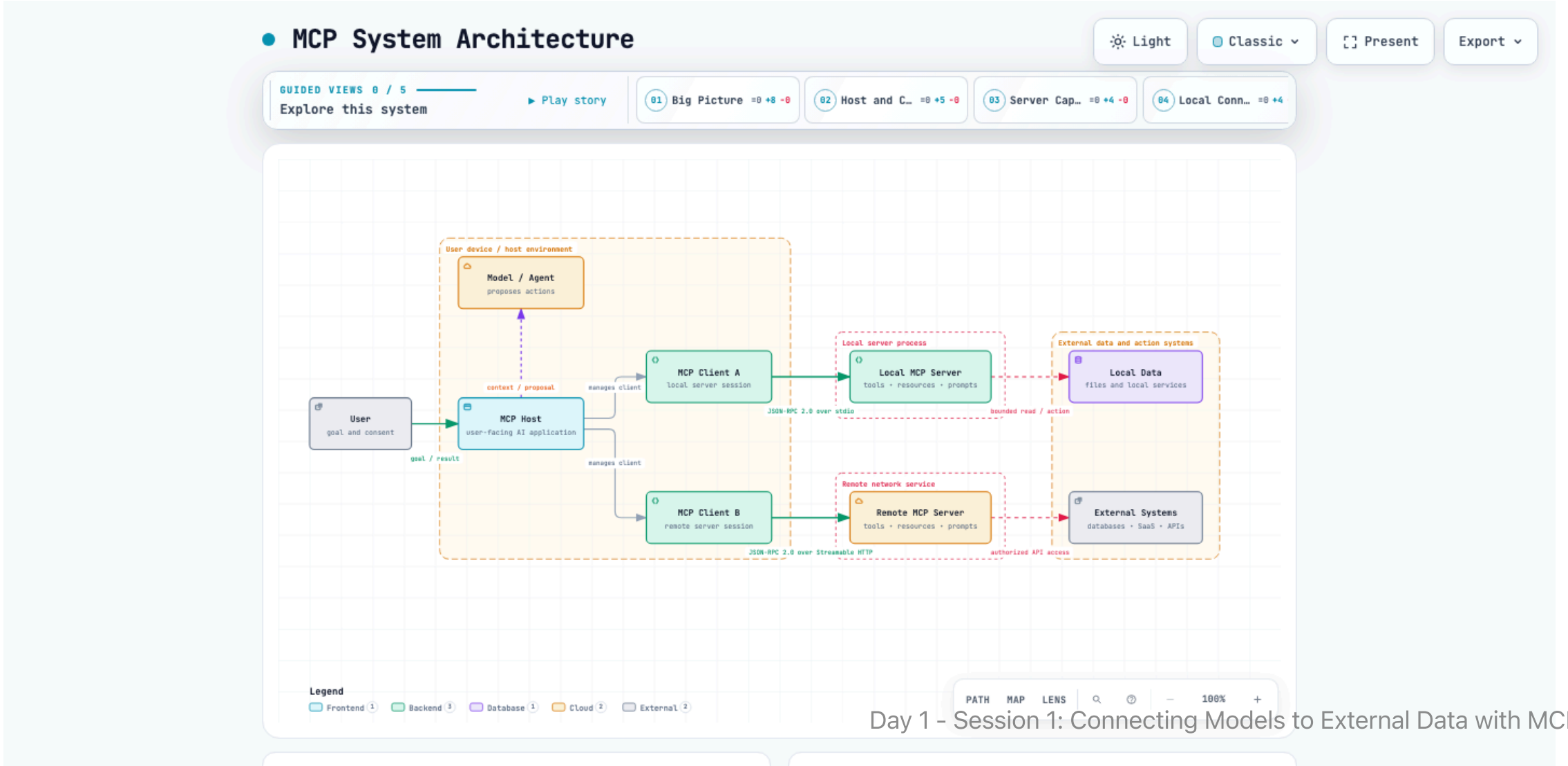
An MCP integration has distinct roles and boundaries.

- **Host**: Your application (e.g., Claude Code, VS Code Copilot)
- **Client**: Connects the Host to the Server
- **Server**: Exposes tools, resources, and prompts
- **Model**: Proposes the actions

MCP Architecture



Interactive: MCP System Architecture



MCP Architecture (Technology/SaaS)

REAL-WORLD SCENARIO

Technology/SaaS (HubSpot, ChatGPT CRM Connector):

- HubSpot's remote MCP server gives ChatGPT permission-aware CRM access for more than 250,000 businesses.
- Its gateway discovers annotated service methods and registers them as MCP tools, separating connection, discovery, and execution.

Source: HubSpot Engineering

MCP Architecture (Manufacturing)

SKIP | REAL-WORLD SCENARIO

Manufacturing (PTC, ThingWorx MCP Preview):

- The ThingWorx AI Assistant acts as an MCP client while each plant or regional ThingWorx instance acts as a server.
- Site servers expose tools, prompts, and resources backed by ERP, MES, PLM, and connected industrial equipment.

Source: PTC ThingWorx documentation

Connecting Models to External Data

Transports and configurations

KavinSchool

Transport Mechanisms

Servers and Clients communicate via transports.

- **Standard input/output (`stdio`)**: Used for local servers. Process isolation and local OS permissions apply.
- **Streamable HTTP**: Used for remote servers. Allows API Gateways, Transport Layer Security (TLS), and network routing.

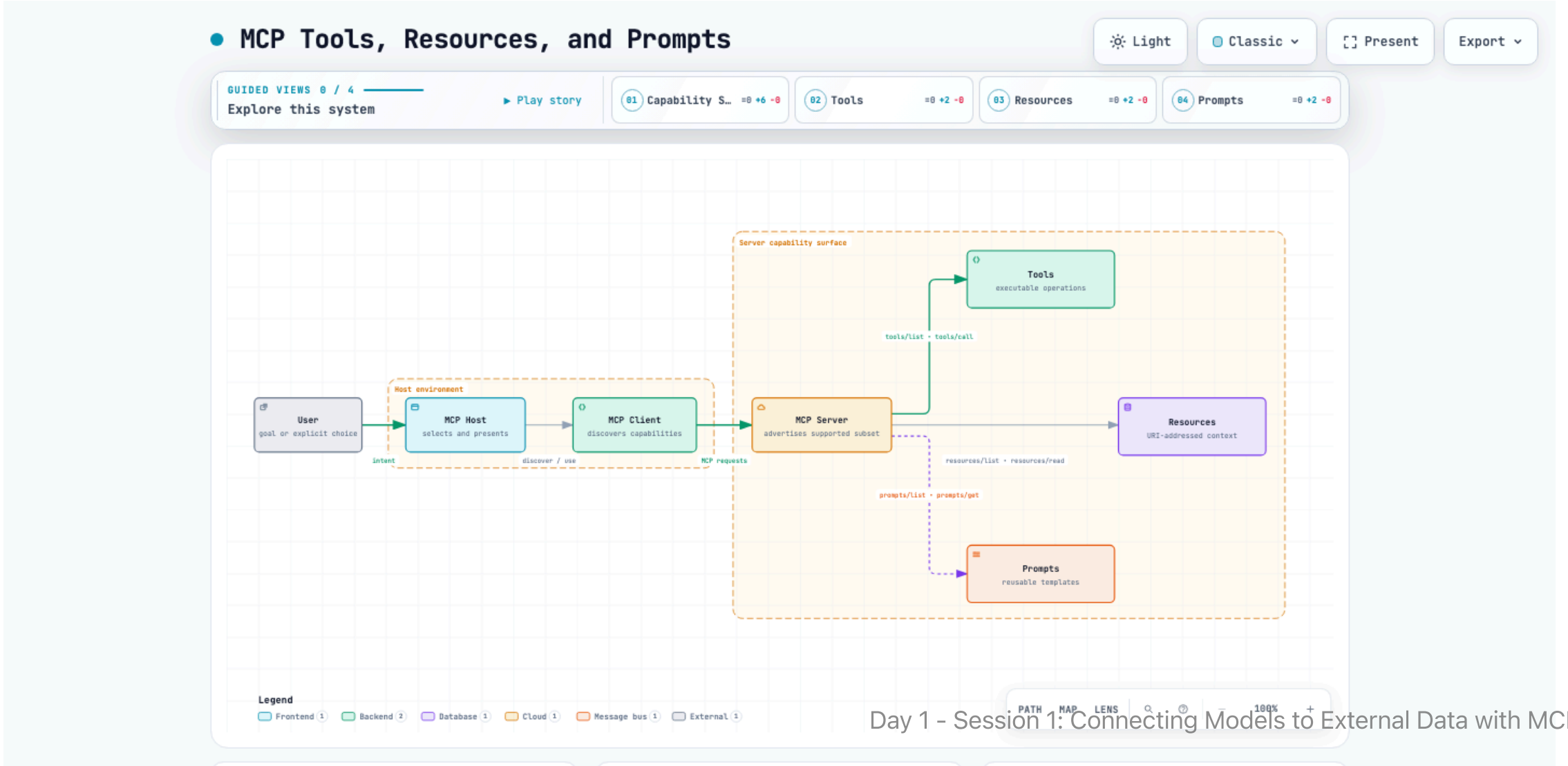
> **NOTE**
> Legacy HTTP with Server-Sent Events (SSE) is deprecated in new MCP 2026-07-28 implementations.

Capabilities

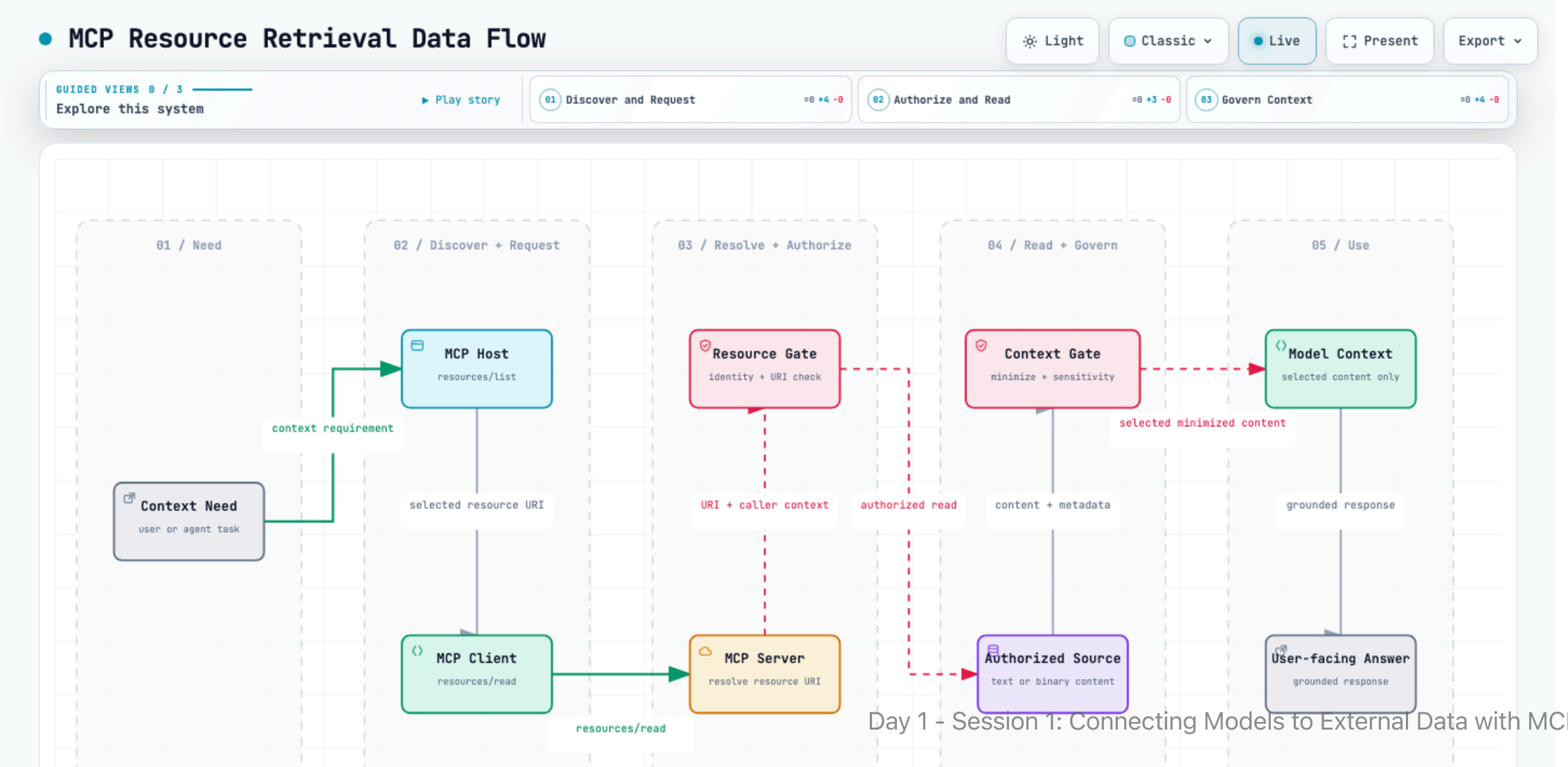
Servers expose three primary types of capabilities:

1. **Tools**: Model-selectable actions (e.g., "query database")
2. **Resources**: Read-only data identified by Uniform Resource Identifiers (URIs)
3. **Prompts**: User-invoked templates

Interactive: Tools, Resources, and Prompts



Interactive: Resource Retrieval



Host Environments

As developers, you typically interact with MCP through an **AI Host** that acts as your client.

- **Claude Code**: A terminal-based assistant that runs locally. Excellent for CLI workflows and scripting.
- **GitHub Copilot in VS Code**: A GUI-integrated assistant within your IDE. Provides a visual chat interface for managing and using MCP servers.

Managing MCP in Claude Code

In Claude Code, you manage your MCP server connections directly via terminal commands.

- `claude mcp list`: View all configured MCP servers.
- `claude mcp add <name> <command> [args...]`: Register a new MCP server.
- `claude mcp remove <name>`: Delete an MCP server configuration.

> Configuration is stored locally and read whenever Claude Code starts.

Configuration Scope

Claude Code stores servers in three scopes, all managed through `claude mcp add -s <scope>`.

Scope	Shared?	Use for
local (default)	No, private to you in this project	Personal creds pointed at one repo
project	Yes, committed via `.mcp.json`	Shared team tooling (e.g., Playwright)
user	No, but available in every project	Personal tools you always want

Precedence when a name exists in multiple scopes: local > project > user.

> Run `claude mcp get <name>` to see exactly which scope and file is in effect.

Trap: Editing the Wrong Config File

The most common MCP debugging mistake is not a broken server. It is an edit that silently lands in the wrong file.

- A higher-precedence scope can shadow an edit made in a lower one, so nothing appears wrong until you actually call the tool.
- A hand-made config file can look exactly like a real scope file (same shape, same folder pattern) while never being read by the client at all. Restarting the IDE or CLI will not fix this, since the file it re-reads was never the stale one.

> **WARNING**
>
> Never assume a file is authoritative because of where it sits or how it looks. Ask the tool: `claude mcp get <name>` names the exact scope and storage path in effect, every time.

Managing MCP in VS Code

In GitHub Copilot for VS Code, you manage MCP servers via JSON configuration and the Command Palette.

- **Configuration:** Define servers in a `.vscode/mcp.json` file at the workspace root, scaffolded via **MCP: Open Workspace Configuration**, or in **MCP: Open User Configuration** for all workspaces.
- **Starting Servers:** Use **MCP: List Servers** to view status; if a server seems stuck, use **MCP: Restart Server**.
- **Agent Mode:** Open Copilot Chat in Agent mode and ensure the intended tools are discovered and enabled.

> The workspace file registers the server command with the VS Code MCP client.

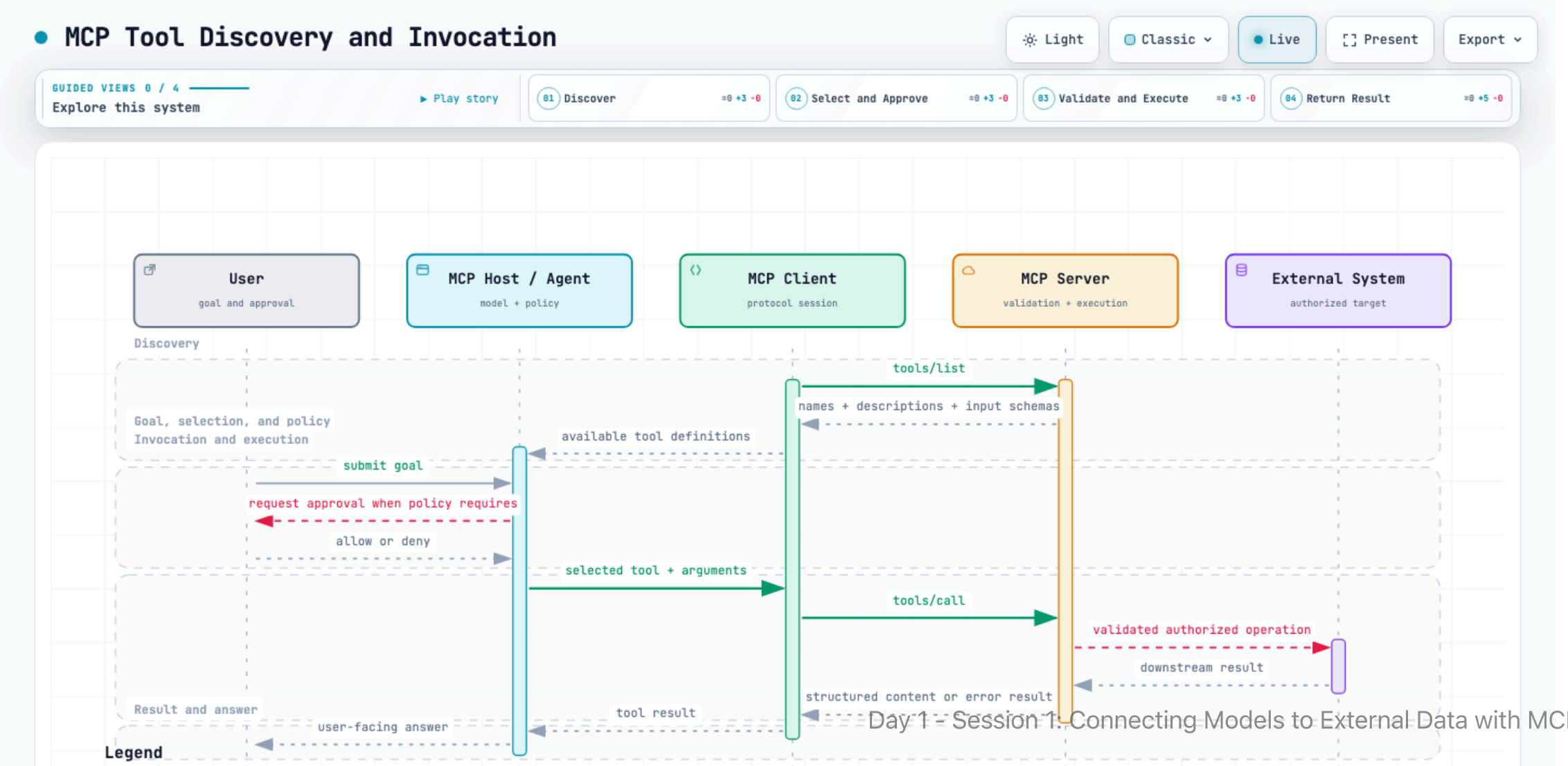
Client Configuration and Trust

Clients use a configuration file (`mcp.json` or `claude_desktop_config.json`) to remember servers and manage trust.

- **Configuration:** Defines the server name, command (e.g., `node` , `python`), and required arguments.
- **Trust & Approval:** Clients prompt for user approval the first time a tool is invoked.
- **Auto-Approval:** You can choose to "Always Allow" specific tools, shifting the responsibility from manual review to the server's internal logic.

> **Important:** Never auto-approve tools that modify state or execute arbitrary code without strict server-side safeguards.

Interactive: Tool Discovery and Invocation



Live Demo: Playwright MCP Configuration

Let's connect the **Playwright MCP Server** to perform web automation and testing.

VS Code (`.vscode/mcp.json`):

```
{
  "servers": {
    "playwright": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-playwright"]
    }
  }
}
```

Claude Code CLI:

```
claude mcp add playwright -- npx -y @modelcontextprotocol/server-playwright
```

Live Demo: Playwright MCP Execution

Once connected, the model discovers Playwright tools and can drive the browser automatically.

Example Prompt:

```
Navigate to https://kavinschool.com/. Explore the homepage, identify the main navigation links, and take a screenshot of the homepage.
```

Capabilities we'll observe:

- **Navigating** to URLs and interacting with UI elements
- **Taking screenshots** of the final page state

> This demonstrates a powerful "no-code" interaction pattern, allowing the AI to "see" and interact with the web directly.

Demo: Pre-built Read-only Server

```
npm --prefix day1/prebuilt-readonly-server run verify
```

- **Fallback signals:** one discovered tool, exact synthetic result, and denied path input
- **Live-host signals:** model proposal and the installed host's approval behavior
- **Duration:** 7 minutes
- **Fallback:** The command above proves MCP discovery and invocation without simulating host UI.

> Client configuration registers how to start the server; the server exposes one fixed read-only tool.

Troubleshooting MCP Connections (1/2)

Symptom	Likely Cause	Fix
Edited config, nothing changed	Edited a lower-precedence scope	`claude mcp get <name>` to find the scope in effect, then edit/re-add there
Server shows "Needs authentication"	HTTP/SSE server using OAuth, no token yet	Trigger the client's own sign-in flow (`/mcp` in Claude Code, or the tools UI prompt in VS Code)

Troubleshooting MCP Connections (2/2)

Symptom	Likely Cause	Fix
Connected, but tool calls still fail	Transport is fine; the server's downstream dependency is unreachable	Read the tool-call error, not the connection status
Secret visible in a config file	Config files are plain JSON, no encryption	Keep client config dirs out of git; prefer `${ENV_VAR}` substitution

> When in doubt, trust `claude mcp get <name>` over your assumption of where config lives.

Kahoot Quiz 1

Check your understanding of these topics.

Please join the Kahoot session on your devices.

- MCP Architecture
- Pre-built Server Configuration
- Transport Selection
- Capability Discovery

Key Takeaways

1. MCP separates the Host, Client, Server, and Model into distinct boundaries.
2. The current specification (`2026-07-28`) introduces self-describing stateless requests.
3. Local labs use `stdio` while remote production systems use Streamable HTTP.
4. Clients configure the connection, but Servers define the tools exposed.

Questions?

Up Next: Custom MCP Servers and Security

KavinSchool